



Universidad Pontificia de Salamanca

Práctica final grupal:

Plataforma de gestión para un centro cultural.

Asignatura: Arquitectura del Software

Alumnos:

Barreras Sierra, Gaspar

Castro Esteban, Mario

Egido Vaquero, Bruno

Fecha: 11 de mayo de 2026

Parte teórica

1. Diseño arquitectónico

La arquitectura del sistema se ha diseñado teniendo en cuenta tanto los requisitos definidos en el enunciado como las características propias de una aplicación de gestión de este tipo. En primer lugar, se ha optado por un sistema centralizado en el que únicamente el administrador puede acceder al sistema y realizar operaciones sobre actividades, usuarios, monitores y salas. Esta decisión simplifica la gestión y encaja con el enfoque planteado en la práctica, donde los usuarios inscritos no interactúan directamente con la aplicación.

Debido a este enfoque centralizado, el sistema se desplegaría localmente en el ordenador del administrador del centro cultural. El acceso se realizaría mediante un navegador web conectado a la aplicación a través de un puerto preconfigurado. Además, podría definirse un alias local para simplificar el acceso y evitar que el administrador tuviese que introducir manualmente la dirección y el puerto cada vez que quisiera utilizar el sistema.

Para la implementación se ha escogido Django, el *framework* visto durante la asignatura. Esta tecnología proporciona una estructura muy adecuada para aplicaciones orientadas a la gestión de información y operaciones CRUD. Además, integra de forma nativa mecanismos como el sistema de rutas, el motor de plantillas, los formularios y el ORM, permitiendo desarrollar la aplicación de manera rápida y manteniendo una organización clara del código.

A nivel interno, el sistema se ha dividido en distintos componentes según su responsabilidad, separando las vistas, los formularios, los *templates* y los modelos. Esta organización modular facilita el mantenimiento y hace que cada parte tenga una función claramente definida.

Por último, se ha utilizado SQLite como base de datos debido a su simplicidad y a que resulta suficiente para el alcance de esta práctica académica, evitando configuraciones adicionales innecesarias durante el desarrollo.

2. Patrones de diseño

El patrón de diseño *Adapter* puede resultar especialmente conveniente dentro de nuestra arquitectura, ya que la propia naturaleza de esta implica depender de interfaces externas sobre las que no tenemos control directo.

En concreto, el *framework* Django proporciona su propia manera de representar la validación de datos, lo que acopla a nuestra aplicación a una interfaz concreta definida por el *framework*. El patrón *Adapter* aparece como solución, ofreciendo a la aplicación una interfaz propia e independiente de cómo Django gestione internamente sus recursos.

Problema identificado

Dentro del sistema, observamos que la validación de los datos introducidos a través de los formularios se apoya en el mecanismo nativo que Django proporciona mediante los formularios (`forms.py`). Este mecanismo posee su propia estructura para representar y generar errores, utilizando elementos como `form.errors`, `ValidationError` y mensajes dependientes del renderizado por defecto en pantalla.

Depender directamente de esta representación nativa genera un fuerte acoplamiento entre la capa de presentación y el comportamiento interno de Django. Cada vez que se quisiera modificar la forma en la que se muestran los errores al usuario, estandarizar su formato o personalizarlos según el contexto, sería necesario intervenir en múltiples puntos del sistema. Esto dificulta el mantenimiento y limita progresivamente el control sobre la presentación de la información.

Solución propuesta

Para paliar el problema de acoplamiento detectado, se propone el uso del patrón *Adapter* como una solución estructural que actúa como traductor entre interfaces incompatibles. Su función principal consiste en permitir que la aplicación trabaje con una interfaz propia y coherente con sus necesidades,

evitando depender directamente de los formatos impuestos por el *framework*.

En nuestra arquitectura, este patrón se aplicaría tratando el sistema de validación nativo de Django —junto con objetos como `form.errors` y `ValidationError`— como el componente externo que debe adaptarse, encapsulando su complejidad interna.

Bajo este esquema, el *Adapter* se define como una clase intermedia encargada de transformar la salida nativa de Django en una estructura de datos propia y estandarizada por la aplicación. Este proceso de traducción permite que cualquier cambio interno del *framework* quede aislado dentro del adaptador, protegiendo al resto del sistema frente a modificaciones imprevistas.

Finalmente, esta aproximación permitiría que la capa de presentación, representada en este caso por `formulario_registro.html`, consuma una interfaz uniforme y predecible. Al desacoplar la visualización de errores de la lógica interna de Django, el *frontend* recibiría siempre el mismo formato de información, facilitando tanto el mantenimiento estético como funcional del sistema.

3. MVC vs. MVT

El patrón **MVC** (Modelo-Vista-Controlador) se ha consolidado como uno de los modelos arquitectónicos más utilizados en el desarrollo web gracias a su capacidad para separar las responsabilidades en tres componentes claramente diferenciados. En esta estructura, el **Modelo** se encarga de la gestión de los datos y de la lógica de negocio en comunicación directa con la base de datos, manteniéndose independiente de la visualización. Por su parte, la **Vista** se limita a mostrar la información al usuario, mientras que el **Controlador** actúa como intermediario, gestionando las peticiones y coordinando el flujo de datos entre los distintos componentes.

Sin embargo, Django adopta una filosofía ligeramente distinta basada en el patrón **MVT** (Modelo-Vista-*Template*). Aunque el **Modelo** mantiene la misma responsabilidad de gestionar los datos y la lógica del sistema, la principal diferencia se encuentra en cómo se distribuyen las tareas relacionadas con la

presentación y el procesamiento de las peticiones. En Django, la **Vista** se encarga de procesar la lógica necesaria y decidir qué información debe mostrarse, mientras que la Plantilla (**Template**) se ocupa exclusivamente de la representación visual de los datos.

La diferencia principal entre ambos modelos aparece al comparar el papel del controlador. En el modelo MVC tradicional, el Controlador centraliza la lógica de interacción entre el usuario, la vista y el modelo. En cambio, en Django estas responsabilidades se encuentran repartidas entre el sistema de rutas, las vistas y otros componentes internos del *framework*. De este modo, la Vista tradicional de MVC queda representada funcionalmente por las plantillas, que únicamente renderizan la información recibida sin conocer cómo se ha obtenido.

Ejemplo práctico

Para comprender mejor cómo se materializa el patrón MVT en nuestro proyecto, resulta útil analizar una petición real. Tomando como ejemplo el acceso del administrador a la ruta `/home/actividades/`, el proceso comienza cuando el URL *router* recibe la petición y la redirige hacia la función `listar_actividades()` ubicada en `actividades_view.py`. Este componente actúa como punto de entrada, relacionando cada URL con la lógica correspondiente.

Una vez que la Vista recibe el control, comienza el procesamiento de la petición extrayendo los parámetros necesarios, como posibles filtros enviados mediante parámetros GET. En este punto, la vista interactúa con el Modelo utilizando el ORM de Django mediante instrucciones como `Actividad.objects.filter(...)`. Este mecanismo permite consultar la base de datos de forma segura y eficiente, evitando la necesidad de escribir consultas SQL directamente.

Finalmente, tras obtener y procesar los resultados, la Vista envía los datos a la plantilla `lista_actividades.html`. Este componente se encarga exclusivamente de renderizar la información en formato HTML para devolverla

al navegador del usuario. Esta separación de responsabilidades permite que el Modelo permanezca desacoplado de la presentación, que la Vista se centre en la lógica de procesamiento y que la Plantilla únicamente gestione la representación visual de los datos, facilitando así el mantenimiento y la organización del sistema.

Parte práctica

- URL del repositorio en Github:

https://github.com/GasparB-21/PracticaGrupal-AS_GestionCentroCultural.git

En el modelado del sistema se han tomado algunas decisiones para adaptar el enunciado a una aplicación sencilla y coherente con el alcance de la práctica. Por ejemplo, el campo `especializacion` de un monitor utiliza la misma enumeración que el campo `tipo` de una actividad, ya que parecía lo más lógico para mantener una clasificación uniforme entre monitores y actividades. Además, se ha considerado que el **responsable** de una sala sea directamente un `Monitor`, evitando así añadir una entidad nueva al modelo de datos y manteniendo la estructura más simple.

También se ha decidido no añadir restricciones adicionales sobre la capacidad de las salas, ya que el enunciado no indica ninguna regla concreta relacionada con este campo. En cuanto a los monitores, no se permite eliminarlos si tienen actividades asociadas o si son responsables de alguna sala. Para que esto no bloquee al administrador sin explicación, se ha añadido una pantalla desde la que puede acceder directamente a la edición de los elementos relacionados y corregir la situación antes de continuar con la eliminación.

Por otro lado, aunque el enunciado plantea una relación 1 a 1 entre sala y responsable técnico, se ha modelado como una relación opcional. De este modo, una sala puede no tener responsable asignado y un monitor puede no

ser responsable de ninguna sala. Esta decisión resulta más realista, ya que en un centro cultural no tiene por qué existir siempre el mismo número de salas que de monitores.

Por último, para comprobar el correcto funcionamiento de los modelos, relaciones y reglas de dominio, se han generado una serie de tests con apoyo de IA.